UTM

UNIVERSITI TEKNOLOGI MALAYSIA

UNIVERSITI PENYELIDIKAN

SECJ2013

DATA STRUCTURE AND ALGORITHM

SECTION 02

ASSIGNMENT 2

LECTURER:

DR. ZURAINI BINTI ALI SHAH

GROUP 3 MEMBER:

LIM YU HAN (A23CS0241)

EVELYN GOH YUAN QI (A23CS0222)

ANIS SAFIYYA BINTI JANAI (A23CS0049)

PRAVINRAJ A/L SIVABATHI (A23CS0171)

Table of Contents

Task 1

```cpp
#include <iostream>
using namespace std;

const int size = 10;  // Maximum size of stack

class Stack {
    int arr[size];
    int top;

public:
    Stack()
    {
        top = -1;
    }

    void push(int x)   // Adds an element to the top of the stack
    {
        if (isFull())
        {
            cout << "Sorry, cannot push item. Stack is now full!\n";
            return;
        }
        else
        {
            top++;
            arr[top] = x;
        }
    }

    void pop()   // Removes the top element of the stack
    {
        if (isEmpty())
        {
            cout << "Sorry, cannot pop item. Stack is empty!\n";
            return;
        }
        else
        {
            cout << "Popped value: " << arr[top] << endl;
            top--;
        }
    }
```

```
    int stackTop()  // Returns the top element without removing it
    {
        if (isEmpty())
        {
            cout << "Stack is empty\n";
            return -1;
        }
        else
            cout << "Top value in the stack is: " << arr[top] << endl;
    }

    bool isEmpty()  // Checks if the stack is empty
    {
        return (top == -1);
    }

    bool isFull()  // Checks if the stack is full (for array-based implementation)
    {
        return (top == size - 1);
    }
};
```

LIFO (Last-In-First-Out) principle

The stack operates on the LIFO principle, in which the last element added is the first element removed. The LIFO principle in stacks is applicable in a variety of real-world scenarios. For example, stacks are important in applications such as browser navigation, where the most recently seen webpage can be popped to return to the previous one. Furthermore, stacks are frequently used in computational processes like recursive algorithm calls to ensure that the last function call is processed first.

Task 2

1. Practical Application: Undo/Redo in Text Editors

The Undo/Redo feature is crucial in text editors, including but not limited to Microsoft Word and Notepad. It allows users to backtrack and restore the previous state or recover it in case something goes wrong during editing.

How it Works:

- Undo Stack:
  - You push an action onto the undo stack whenever you do something, say, typing a word or deleting a sentence.
  - Pressing "Undo" pops the most recent action off the stack, reverses it-for example, removes the typed word or restores the deleted sentence-and pushes it onto the redo stack for possible redoing.

- Redo Stack:
  - Upon the press of "Redo," the most recently undone action is popped off the redo stack, reapplied, and moved back to the undo stack.
  - The above arrangement ensures that the actions would be treated in the correct order, maintaining consistency and order.

2. Key Advantages of Using Stacks in Undo/Redo Functionality

1. LIFO Nature

The LIFO nature of a stack guarantees that the most recent action is always the first to be undone. To put this into perspective, in a case where a user types a word and deletes it, the delete action is the last one recorded on the stack. It becomes the first action to be reversed once the undo operation is triggered; hence, the word deleted gets restored. This will also be very intuitive for the user and logical since actions must be undone in reverse order of their execution.

2. Isolation of Undo and Redo Operations

Using two different stacks, one for undo and one for redo, allows the operations to be treated clearly without conflict. When an action is undone, it is removed from the undo stack and pushed onto the redo stack. Similarly, when an action is redone, it is moved back from the redo stack to the undo stack. This separation will enable the system to manage both operations independently, assuring a seamless transition between the states of undo and redo without any overwriting or loss of data.

3. Efficiency

Stacks are inherently efficient because their push and pop operations take constant time, O(1), meaning thereby that the time to push or pop an action is independent of the number of actions stored on the stack. Consequently, even with a large history of operations, this implementation of the undo/redo functionality executes in effectively constant time. This efficiency becomes important for maintaining a seamless user experience, especially when responsiveness is paramount in an application.

4. Example

When the user types "Hello," the action "Type 'Hello'" is pushed onto the undo stack. If the user hits "Undo," this action is popped from the undo stack, reversed and the text is removed and then pushed onto the redo stack. Later, if the user hits "Redo," the action is popped from the redo stack, reapplied, the text is restored-and then pushed back onto the undo stack. This smooth transition between the stacks ensures that the undo/redo feature works as it should.

Task 3

Array-Based Implementation

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX_SIZE = 100; // Maximum size of the stack

class Stack {
private:
    string data[MAX_SIZE]; // Array to store stack elements
    int top;               // Index of the top element

public:
    Stack() : top(-1) {} // Constructor initializes the stack as empty

    bool isFull() {
        return top == MAX_SIZE - 1;
    }

    bool isEmpty() {
        return top == -1;
    }

    void push(string item) {
        if (isFull()) {
            cout << "Stack is full! Cannot push more actions.\n";
        } else {
            top++;
            data[top] = item;
        }
    }

    string pop() {
        if (isEmpty()) {
            cout << "Stack is empty! Cannot pop actions.\n";
            return "";
        } else {
            return data[top--];
        }
    }

    string stackTop() {
        if (isEmpty()) {
            cout << "Stack is empty!\n";
            return "";
        } else {
            return data[top];
        }
    }

    void display() {
        if (isEmpty()) {
            cout << "Stack is empty!\n";
        } else {
            cout << "Stack contents: ";
            for (int i = 0; i <= top; i++) {
                cout << data[i] << " ";
            }
            cout << endl;
        }
    }
};
```

```cpp
int main() {
    Stack undoStack, redoStack;
    string action;
    int choice;

    do {
        cout << "\nUndo/Redo Menu:\n";
        cout << "1. Perform Action (Push to Undo Stack)\n";
        cout << "2. Undo (Move from Undo to Redo Stack)\n";
        cout << "3. Redo (Move from Redo to Undo Stack)\n";
        cout << "4. Display Undo Stack\n";
        cout << "5. Display Redo Stack\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        cin.ignore(); // To clear the input buffer

        switch (choice) {
        case 1:
            cout << "Enter action to perform: ";
            getline(cin, action);
            undoStack.push(action);
            // Clear redo stack after a new action
            while (!redoStack.isEmpty()) {
                redoStack.pop();
            }
            cout << "Action performed: " << action << endl;
            break;

        case 2:
            if (!undoStack.isEmpty()) {
                action = undoStack.pop();
                redoStack.push(action);
                cout << "Undone action: " << action << endl;
            } else {
                cout << "Nothing to undo!\n";
            }
            break;

        case 3:
            if (!redoStack.isEmpty()) {
                action = redoStack.pop();
                undoStack.push(action);
                cout << "Redone action: " << action << endl;
            } else {
                cout << "Nothing to redo!\n";
            }
            break;

        case 4:
            cout << "Undo Stack:\n";
            undoStack.display();
            break;

        case 5:
            cout << "Redo Stack:\n";
            redoStack.display();
            break;

        case 6:
            cout << "Exiting program.\n";
            break;

        default:
            cout << "Invalid choice! Please try again.\n";
        }
    } while (choice != 6);

    return 0;
}
```

Output

```
Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 1
Enter action to perform: hello
Action performed: hello

Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 1
Enter action to perform: world
Action performed: world

Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 4
Undo Stack:
Stack contents: hello world

Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 5
Redo Stack:
Stack is empty!

Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 2
Undone action: world
```

```
Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 4
Undo Stack:
Stack contents: hello

Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 5
Redo Stack:
Stack contents: world

Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 3
Redone action: world

Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 4
Undo Stack:
Stack contents: hello world

Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 5
Redo Stack:
Stack is empty!
```

```
Undo/Redo Menu:
1. Perform Action (Push to Undo Stack)
2. Undo (Move from Undo to Redo Stack)
3. Redo (Move from Redo to Undo Stack)
4. Display Undo Stack
5. Display Redo Stack
6. Exit
Enter your choice: 6
Exiting program.

--------------------------------
Process exited after 50.65 seconds with return value 0
Press any key to continue . . .
```

Pointer-Based Implementation

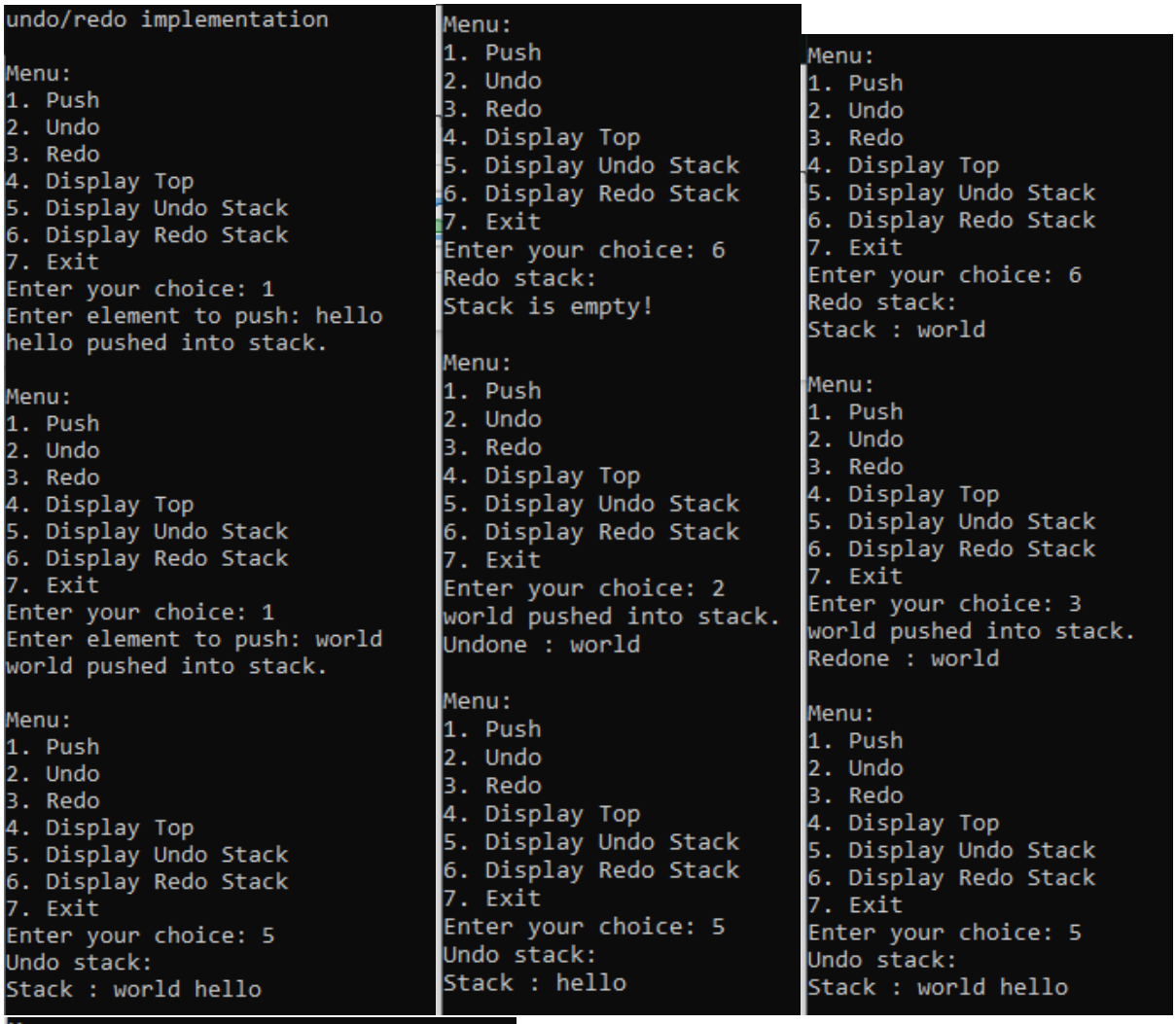
```cpp
#include <iostream>
#include <string>
using namespace std;

class nodeStack {
public:
    string data;
    nodeStack *next;
};

class stack {
private:
    nodeStack *top;
public:
    void createStack() { top = NULL; }
    void push(string);
    string pop();
    string stackTop();
    bool isEmpty() { return (top == NULL); }
    void displayStack();
};

void stack::push(string newitem) {

    nodeStack *newnode = new nodeStack();
    if (newnode == NULL)
        cout << "Cannot allocate memory" << endl;
    else {
        newnode->data = newitem;
        newnode->next = top;
        top = newnode;
        cout << newitem << " pushed into stack." << endl;
```

```cpp
string stack::pop() {
    if (isEmpty()) {
        return "";
    } else {
        nodeStack *delnode = top;
        // cout << "Popped " << stackTop() << endl;
        string pop = stackTop();
        top = delnode->next;
        delete delnode;
        return pop;
    }
}

string stack::stackTop() {
    if (isEmpty()) {
        cout << "Sorry, stack is still empty!" << endl;
        return "";
    } else {
        return top->data;
    }
}

void stack::displayStack() {
    if (isEmpty()) {
        cout << "Stack is empty!" << endl;
    } else {
        cout << "Stack : ";
        nodeStack *current = top;
        while (current != NULL) {
            cout << current->data << " ";
            current = current->next;
        }
        cout << endl;
```

```
int main() {
    stack Stack, redoStack;
    Stack.createStack();
    redoStack.createStack();
    int choice;
    string a;

    cout << "undo/redo implementation\n";

    do {
        cout << "\nMenu:\n";
        cout << "1. Push \n";
        cout << "2. Undo \n";
        cout << "3. Redo \n";
        cout << "4. Display Top \n";
        cout << "5. Display Undo Stack \n";
        cout << "6. Display Redo Stack \n";
        cout << "7. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        cin.ignore();

        switch (choice) {
        case 1:
            cout << "Enter element to push: ";
            getline(cin, a);
            Stack.push(a);
            while (!redoStack.isEmpty()){
                redoStack.pop();
            }
            break;
```

```
        case 2:
            if (!Stack.isEmpty()){
                a = Stack.pop();
                redoStack.push(a);
            cout << "Undone : " << a <<endl;
            }else{
                cout << "Nothing to undo!\n";
            }
            break;
        case 3:
            if (!redoStack.isEmpty()){
                a = redoStack.pop();
                Stack.push(a);
            cout << "Redone : " << a <<endl;
            }else{
                cout << "Nothing to redo!\n";
            }
            break;
        case 4:
            cout<< "Top is "<< Stack.stackTop() << endl;
            break;
        case 5:
            cout << "Undo stack: "<< endl;
            Stack.displayStack();
            break;
        case 6:
            cout << "Redo stack: "<< endl;
            redoStack.displayStack();
            break;
        case 7:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }
    } while (choice != 7);

    system("pause");
    return 0;
}
```

Output


undo/redo implementation
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 1
Enter element to push: hello
hello pushed into stack.
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 1
Enter element to push: world
world pushed into stack.
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 5
Undo stack:
Stack : world hello
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 6
Redo stack:
Stack is empty!
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 2
world pushed into stack.
Undone : world
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 5
Undo stack:
Stack : hello
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 6
Redo stack:
Stack : world
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 3
world pushed into stack.
Redone : world
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 5
Undo stack:
Stack : world hello
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 6
Redo stack:
Stack is empty!
Menu:
1. Push
2. Undo
3. Redo
4. Display Top
5. Display Undo Stack
6. Display Redo Stack
7. Exit
Enter your choice: 7
Exiting...
Press any key to continue . . .

Task 4

Aspect	Array	Pointer
Memory usage	Fixed sized	Flexible, dynamically allocated
Ease of implementation	Easier to implement with less complexity	More complex and prone to error
Scalability	Limited by the predefined array size	Dynamic, can be adjust as needed
Speed	Faster due to direct indexing	Slower due to indirect referencing
Error Handling	Stack overflow	Dangling pointers, memory leaks, or segmentation faults

Summary Report: Comparative Analysis of Stack Implementations

In this assignment, the array-based and pointer-based implementation of the stack data structure was explored. Each methodology has certain strengths and weaknesses that make either one better suited for any given scenario.

The array-based implementation utilizes a fixed-size array to store the elements in the stack. Its simplicity and ease of implementation make it particularly suitable for applications where the maximum size of the stack can be known beforehand. The operations Push and Pop are pretty efficient because of direct indexing. However, this is a very erroneous approach because memory may be wasted if the stack is not utilized to its full capacity and overflow if the size that was pre-estimated is surpassed. This makes it poorly scalable in dynamic or large-scale applications.

The pointer-based implementation uses dynamic memory allocation with a linked list. This method excels in scenarios where stack size is unpredictable, as it dynamically adjusts to the data's requirements, avoiding memory wastage. Additionally, it eliminates the risk of stack overflow. However, the complexity of managing pointers increases the likelihood of errors, such as memory leaks. Operations in a pointer-based stack are slightly slower due to the overhead of pointer traversal and memory allocation.

The pointer-based stack is better in terms of memory consumption, since it allocates the memory dynamically. Whereas, it is more annoying to be implemented compared with straightforward array-based implementation.

The array-based stack is adequate for a small, fixed-size application. In contrast, the pointer-based stack is much more suitable to handle dynamic usage in any applications. Which implementation to be used depends on scalability demand, memory constraints, or ease of development in one's application.